



King Abdul Aziz University

Faculty of Computing & Information Technology

Computer Science Department



CPCS 212 - APPLIED MATH FOR COMPUTING

Interpolation and Curve fitting

Instructor Dr. Ahmed Harbaoui

Outline

- Polynomial Approximation
- Interpolation and Curve fitting
- Interpolation
 - Direct interpolation
 - Newton basis
- Matlab's built-in interpolation functions

Introduction

In many problems in engineering and science, the data being considered are known only at a set of discrete points, not as a continuous function.

For example, the continuous function

$$y = f(x)$$

may be known only at n discrete values of x :

$$y_i = y(x_i) \text{ for } i = 1, 2, 3, \dots, n$$

x_i	y_i
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
x_n	y_n

Introduction

- ❑ In many applications, the values of the discrete data at the specific points are not all that is needed. **Values of the function at points other than the known discrete points may be needed** (i.e., interpolation).
- ❑ The derivative of the function may be required (i.e., differentiation). The integral of the function may be of interest (ie, integration). Thus, the processes of interpolation, differentiation, and integration of a set of discrete data are of interest.
- ❑ These processes are performed by **fitting** an **approximating** function to the set of discrete data and performing the desired process on the approximating function.

Approximating functions

- ❑ Many types of **approximating functions** exist. In fact, any analytical function can be used as an approximating function. Three of the more common approximating functions are:

1. Polynomials
2. Trigonometric functions
3. Exponential functions

- ❑ Approximating functions should have the following properties:

1. The approximating function should be easy to determine.
2. It should be easy to evaluate.
3. It should be easy to differentiate.
4. It should be easy to integrate.

- ❑ **Polynomials** satisfy all four of these properties. Consequently, polynomial approximating functions are used in this book to fit sets of discrete data for interpolation, differentiation, and integration.

Polynomial Approximation

The general form of an n^{th} degree polynomial is

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n$$

Where

n : denotes the degree of the polynomial

a_i : are constant coefficients

There are $n + 1$ coefficients, so $n + 1$ discrete data points are **required** to obtain unique values for the coefficients

Interpolation vs. Curve Fitting

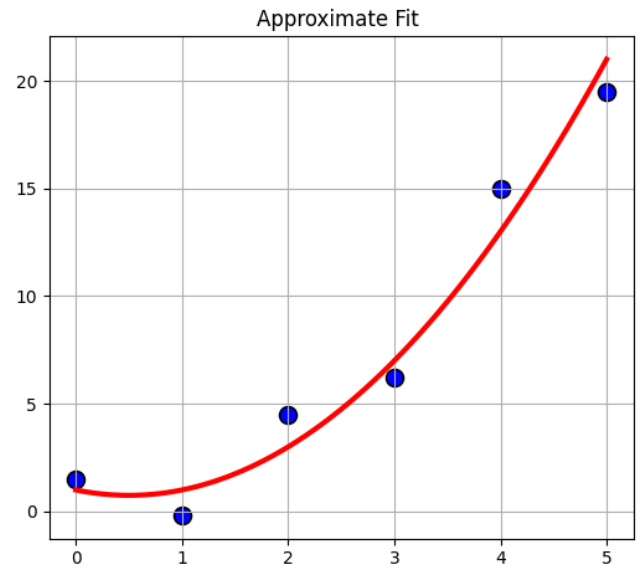
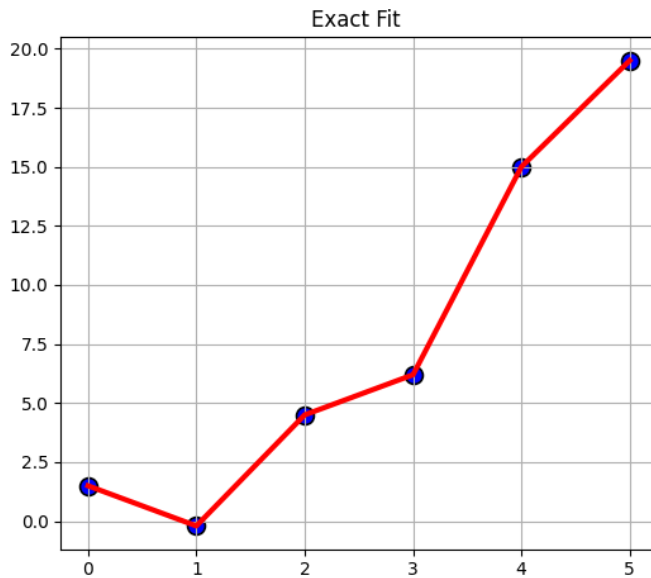
- ❑ There are two fundamentally different ways to fit a polynomial to a set of discrete data:

1. Exact fits  Interpolation

2. Approximate fits  Curve fitting

- ❑ An **exact fit** yields a polynomial that passes exactly through all of the discrete points, This type of fit is useful for small sets of smooth data.
- ❑ An **approximate fit** yields a polynomial that passes through the set of data in the best manner possible, without being required to pass exactly through any of the data points.

Interpolation vs. Curve Fitting



Interpolation: Direct Method

Given 'n+1' data points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, pass a polynomial of order 'n' through the data as given below:

$$y = a_0 + a_1x + \dots + a_nx^n.$$

where $a_0, a_1, \dots, a_n$ are real constants.

- Set up 'n+1' equations to find 'n+1' constants.
- To find the value 'y' at a given value of 'x', simply substitute the value of 'x' in the above polynomial.

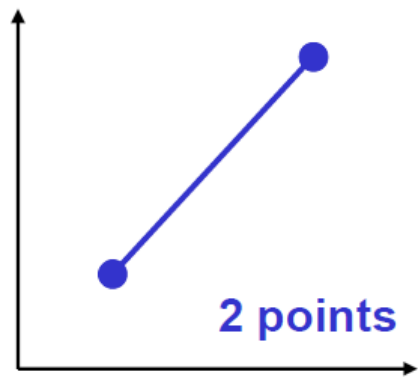
Vandermonde
equations

This representation defines the polynomial in terms of a set of monomial basis functions

$$x^0, x^1, x^2, \dots, x^n$$

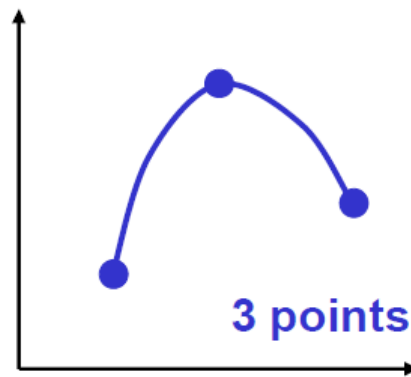
Interpolation: Direct Method

Linear, Quadratic, and Cubic Interpolations



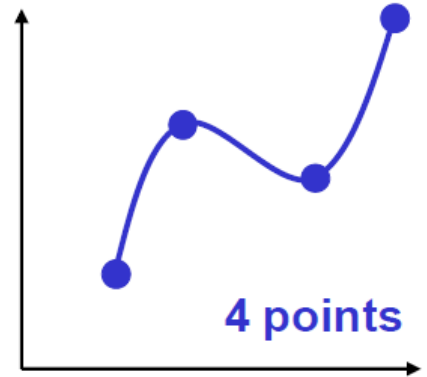
Linear = 1st order

$$f(x) = a_0 + a_1x$$



Quadratic = 2nd order

$$f(x) = a_0 + a_1x + a_2x^2$$



Cubic = 3rd order

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

Interpolation: Direct Method

Example1:

The upward velocity of a rocket is given as a function of time in Table 1.

Find the velocity at $t=16$ seconds using the direct method for

- a) linear interpolation.
- b) quadratic interpolation.

$t, (s)$	$v(t), (m/s)$
0	0
10	227.04
15	362.78
20	517.35
22.5	602.97
30	901.67

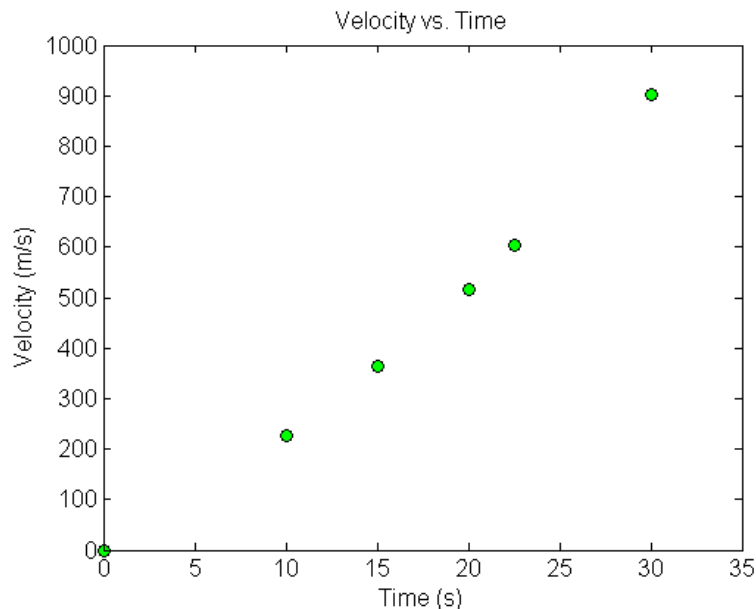


Table Velocity as a function of time

Figure Velocity vs. time data for the rocket example

Example1 : Direct Method (linear Interpolation)

$$v(t) = a_0 + a_1 t$$

$$v(15) = a_0 + a_1(15) = 362.78$$

$$v(20) = a_0 + a_1(20) = 517.35$$

Solving the above two equations gives,

$$a_0 = -100.93 \quad a_1 = 30.914$$

Hence

$$v(t) = -100.93 + 30.914t, \quad 15 \leq t \leq 20.$$

$$v(16) = -100.93 + 30.914(16) = 393.7 \text{ m/s}$$

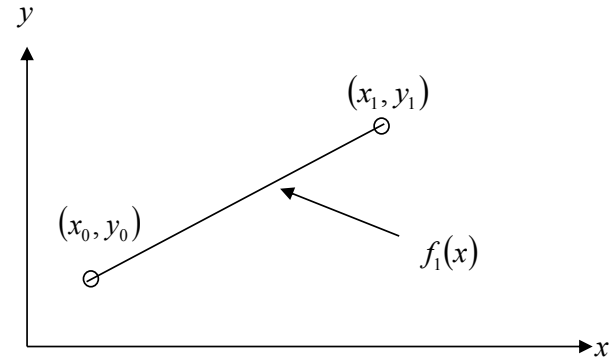


Figure Linear interpolation.

Example1 : Direct Method (quadratic Interpolation)

$$v(t) = a_0 + a_1 t + a_2 t^2$$

$$v(10) = a_0 + a_1(10) + a_2(10)^2 = 227.04$$

$$v(15) = a_0 + a_1(15) + a_2(15)^2 = 362.78$$

$$v(20) = a_0 + a_1(20) + a_2(20)^2 = 517.35$$

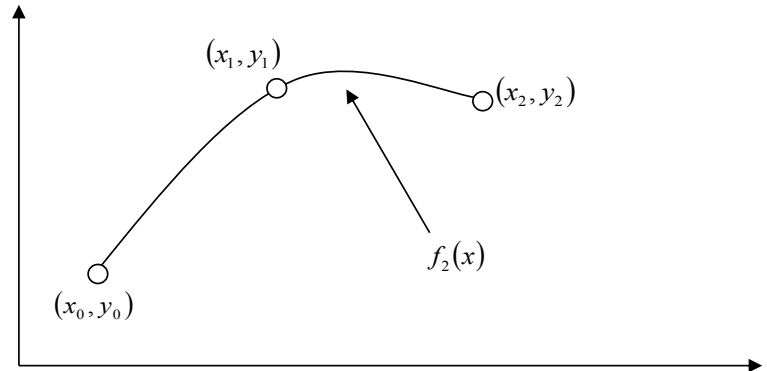


Figure Quadratic interpolation.

Solving the above three equations gives

$$a_0 = 12.05 \quad a_1 = 17.733 \quad a_2 = 0.3766$$

$$v(t) = 12.05 + 17.733t + 0.3766t^2, \quad 10 \leq t \leq 20$$

$$\begin{aligned} v(16) &= 12.05 + 17.733(16) + 0.3766(16)^2 \\ &= 392.19 \text{ m/s} \end{aligned}$$

Example 2. Direct fit polynomials.

To illustrate interpolation by a direct fit polynomial, consider the simple function $y = f(x) = 1/x$, and construct the following set of six significant figure data:

x	$f(x)$
3.35	0.298507
3.40	0.294118
3.50	0.285714
3.60	0.277778

Let's interpolate for y at $x = 3.44$ using linear, quadratic, and cubic interpolation. The exact value is

$$y(3.44) = f(3.44) = \frac{1}{3.44} = 0.290698 \dots$$

Let's illustrate the procedure in detail for a quadratic polynomial:

$$P_2(x) = a + bx + cx^2$$

To center the data around $x = 3.44$, the first three points are used. Applying $P_2(x)$ at each of these data points gives the following three equations:

$$0.298507 = a + b(3.35) + c(3.35)^2$$

$$0.294118 = a + b(3.40) + c(3.40)^2$$

$$0.285714 = a + b(3.50) + c(3.50)^2$$

Solving for a , b , and c by Gauss elimination without scaling or pivoting yields

$$P_2(x) = 0.876561 - 0.256080x + 0.0249333x^2$$

Substituting $x = 3.44$ gives

$$P_2(3.44) = 0.876561 - 0.256080(3.44) + 0.0249333(3.44)^2 = 0.290697$$

The error is $\text{Error}(3.44) = P_2(3.44) - f(3.44) = 0.290697 - 0.290698 = -0.000001$.

For a linear polynomial, use $x = 3.40$ and 3.50 to center that data around $x = 3.44$. The resulting linear polynomial is

$$P_1(x) = 0.579854 - 0.0840400x$$

Substituting $x = 3.44$ gives $P_1(3.44) = 0.290756$. For a cubic polynomial, all four points must be used. The resulting cubic polynomial is

$$P_3(x) = 1.121066 - 0.470839x + 0.0878000x^2 - 0.00613333x^3$$

Substituting $x = 3.44$ into $P_3(3.44) = 0.290698$.

The results are summarized below, where the results of linear, quadratic, and cubic interpolation, and the errors, $\text{Error}(3.44) = P(3.44) - 0.290698$, are tabulated. The advantages of higher-degree interpolation are obvious.

$P(3.44) = 0.290756$	linear interpolation	Error = 0.000058
$= 0.290697$	quadratic interpolation	$= -0.000001$
$= 0.290698$	cubic interpolation	$= 0.000000$

Outline

- Polynomial Approximation
- Interpolation
 - Direct interpolation
 - **Newton basis**
- Curve fitting
- Matlab's built-in interpolation functions

Interpolation: Newton's Divided Difference Method

A *divided difference* is defined as the ratio of the difference in the function values at two points divided by the difference in the values of the corresponding independent variable. Thus, the first divided difference at point i is defined as

$$f[x_i, x_{i+1}] = \frac{f_{i+1} - f_i}{x_{i+1} - x_i}$$

The second divided difference is defined as

$$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]}{x_{i+2} - x_i}$$

Similar expressions can be obtained for divided differences of any order. Approximating polynomials for nonequally spaced data can be constructed using divided differences.

Interpolation: Newton's Divided Difference Method

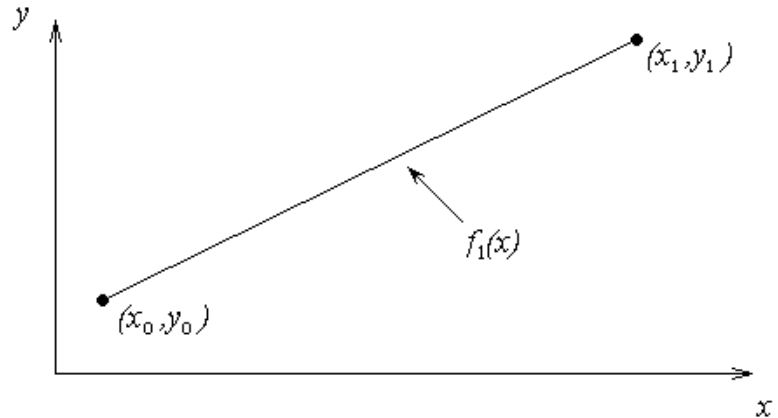
Linear interpolation: Given (x_0, y_0) , (x_1, y_1) , pass a linear interpolant through the data

$$f_1(x) = b_0 + b_1(x - x_0)$$

where

$$b_0 = f(x_0)$$

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$



Interpolation: Newton's Divided Difference Method

Quadratic Interpolation:

Given (x_0, y_0) , (x_1, y_1) , and (x_2, y_2) , fit a quadratic interpolant through the data.

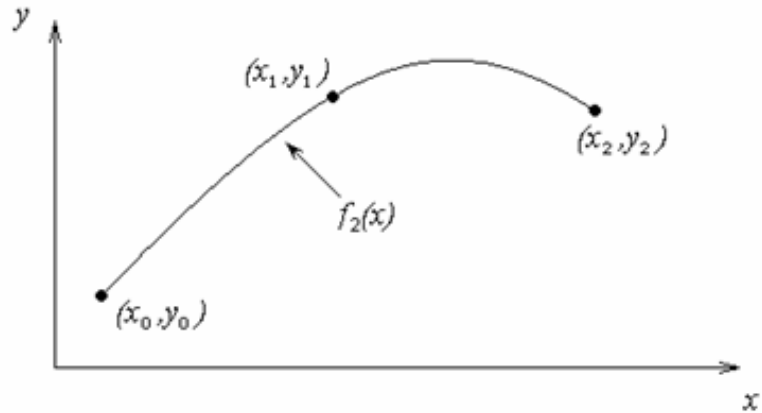
$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$

where

$$b_0 = f(x_0)$$

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$b_2 = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$



Interpolation: Newton's Divided Difference Method

General Form

Given $(n+1)$ data points, $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1}), (x_n, y_n)$ as

$$f_n(x) = b_0 + b_1(x - x_0) + \dots + b_n(x - x_0)(x - x_1)\dots(x - x_{n-1})$$

where

$$b_0 = f[x_0]$$

$$b_1 = f[x_1, x_0]$$

$$b_2 = f[x_2, x_1, x_0]$$

$$\vdots$$

$$b_{n-1} = f[x_{n-1}, x_{n-2}, \dots, x_0]$$

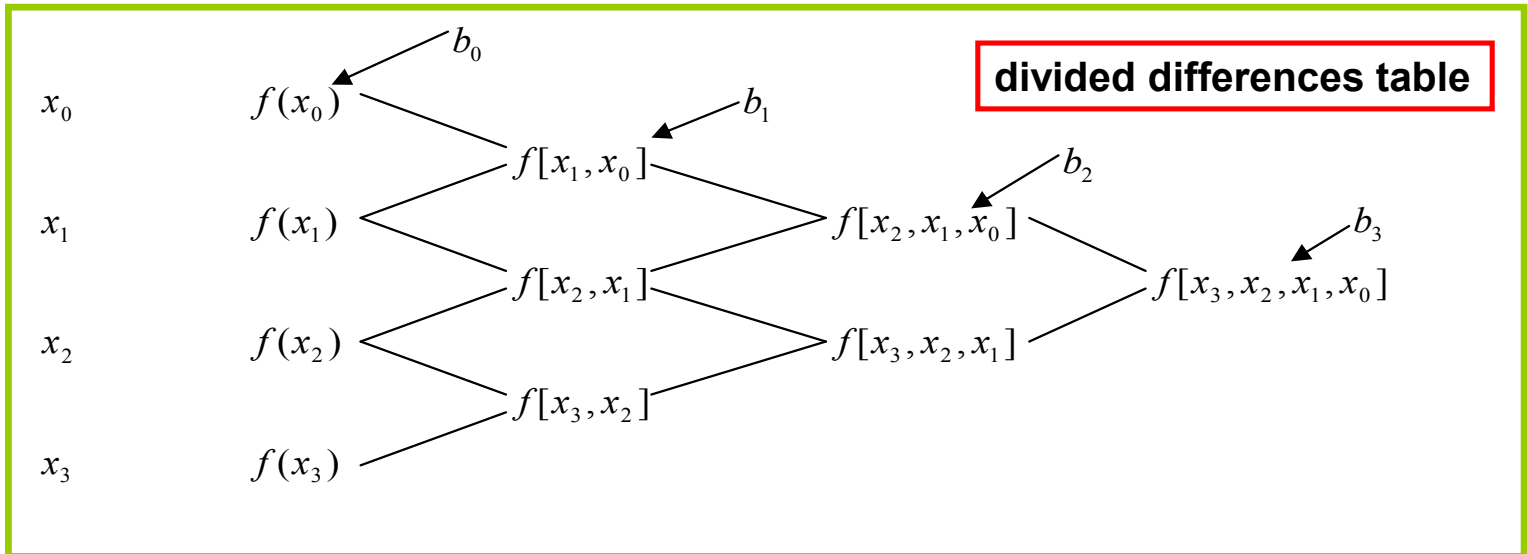
$$b_n = f[x_n, x_{n-1}, \dots, x_0]$$

Interpolation: Newton's Divided Difference Method

Cubic Interpolation:

The third order polynomial, given (x_0, y_0) , (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) , is

$$f_3(x) = f[x_0] + f[x_1, x_0](x - x_0) + f[x_2, x_1, x_0](x - x_0)(x - x_1) + f[x_3, x_2, x_1, x_0](x - x_0)(x - x_1)(x - x_2)$$



Interpolation: Newton's Divided Difference Method

Example 3:

The upward velocity of a rocket is given as a function of time in Table 1. Find the velocity at $t=16$ seconds using the Newton Divided Difference method for:

- a) linear interpolation.
- b) quadratic interpolation.
- c) Cubic interpolation

(s)	(m/s)
0	0
10	227.04
15	362.78
20	517.35
22.5	602.97
30	901.67

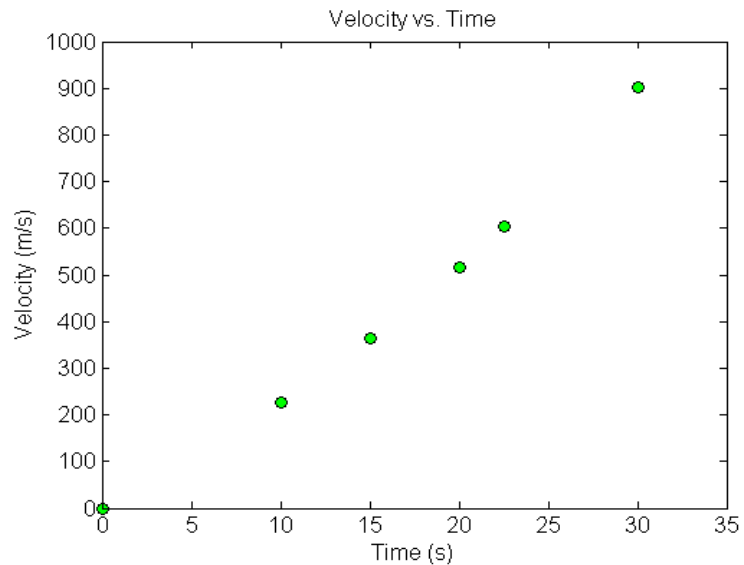


Table: Velocity as a function of time

Figure: Velocity vs. time data for the rocket example

Interpolation: Newton's Divided Difference Method

(linear Interpolation)

$$v(t) = b_0 + b_1(t - t_0)$$

$$t_0 = 15, v(t_0) = 362.78$$

$$t_1 = 20, v(t_1) = 517.35$$

$$b_0 = v(t_0) = 362.78$$

$$b_1 = \frac{v(t_1) - v(t_0)}{t_1 - t_0} = 30.914$$

$$v(t) = b_0 + b_1(t - t_0)$$

$$= 362.78 + 30.914(t - 15), 15 \leq t \leq 20$$

At $t = 16$

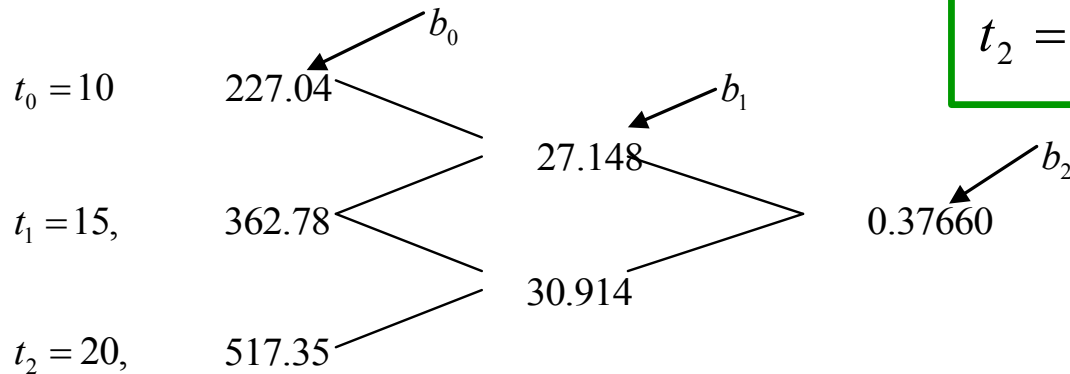
$$v(16) = 362.78 + 30.914(16 - 15)$$

$$= 393.69 \text{ m/s}$$

Interpolation: Newton's Divided Difference Method

(quadratic Interpolation)

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$



$$t_0 = 10, v(t_0) = 227.04$$

$$t_1 = 15, v(t_1) = 362.78$$

$$t_2 = 20, v(t_2) = 517.35$$

$$v(t) = b_0 + b_1(t - t_0) + b_2(t - t_0)(t - t_1)$$

$$= 227.04 + 27.148(t - 10) + 0.37660(t - 10)(t - 15), \quad 10 \leq t \leq 20$$

At $t = 16$,

$$v(16) = 227.04 + 27.148(16 - 10) + 0.37660(16 - 10)(16 - 15) = 392.19 \text{ m/s}$$

Interpolation: Newton's Divided Difference Method

(cubic Interpolation)

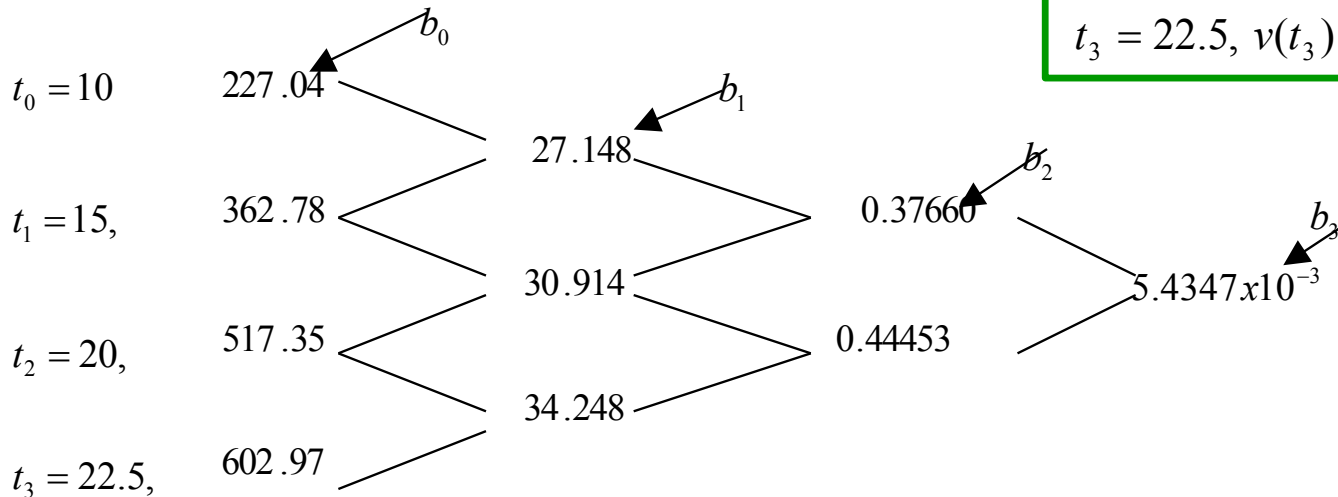
$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$

$$t_0 = 10, \quad v(t_0) = 227.04$$

$$t_1 = 15, \quad v(t_1) = 362.78$$

$$t_2 = 20, \quad v(t_2) = 517.35$$

$$t_3 = 22.5, \quad v(t_3) = 602.97$$



$$b_0 = 227.04; \quad b_1 = 27.148; \quad b_2 = 0.37660; \quad b_3 = 5.4347 \times 10^{-3}$$

Interpolation: Newton's Divided Difference Method

(cubic Interpolation)

Hence

$$\begin{aligned}v(t) &= b_0 + b_1(t - t_0) + b_2(t - t_0)(t - t_1) + b_3(t - t_0)(t - t_1)(t - t_2) \\&= 227.04 + 27.148(t - 10) + 0.37660(t - 10)(t - 15) \\&\quad + 5.4347 * 10^{-3}(t - 10)(t - 15)(t - 20)\end{aligned}$$

At $t = 16$,

$$\begin{aligned}v(16) &= 227.04 + 27.148(16 - 10) + 0.37660(16 - 10)(16 - 15) \\&\quad + 5.4347 * 10^{-3}(16 - 10)(16 - 15)(16 - 20) \\&= 392.06 \text{ m/s}\end{aligned}$$

Matlab's built-in interpolation functions

Polynomial Interpolation (Direct Method)

- ❑ Use **polyfit** to compute a polynomial of degree n that interpolates the data:

```
p = polyfit(x, y, n);           % x, y: data points; n: degree of polynomial  
y_interp = polyval(p, xq);      % xq: query points for evaluation
```

The result is expressed in the **monomial basis**

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n$$

Newton Interpolation (Not Built-in)

- ❑ MATLAB does **not** provide a built-in function for Newton interpolation.
- ❑ A Newton polynomial can be implemented using divided differences in a custom MATLAB function.